

Course: Computer Networks

Name: Kabir Kimtani and Rafsan Ahmed

Student ID: GH1045344 and GH1045023

Lecturer: Dr. Christian Dombrowski

Date of submission: June 24th

Content Table:

1.System design and explanation.

2.Protocol description.

3.Network communication flow chart and finite state machine.

4.Transport layer review and performance metrics.

5.System implementation and environment setup.

1. System design and explanation:

1.1 Design framework selection and explanation:

The application uses a decentralized, localized Client-Server Architecture that manages multi-threaded network communication within private, local networking settings. The main coordinating component handles with connection routing, tracks the system state, and set up chatroom array parameter entirely with on, and create chat room array parameters entirely within the RAM buffer.

- Central Coordinating Node (server.py): functions with a highly optimized multi-threading execution pipeline. A primary listener thread functions a blocking evaluation loop (while true) over port 65432 to catch connection handshakes. As soon as the TCP stream is set up successfully, the connection handle is passed off to an isolated worker thread (client_manag). Active socket pointers and room mappings are stored using core Python dictionary structures, clients and rooms, to avoid the overhead of an external database.

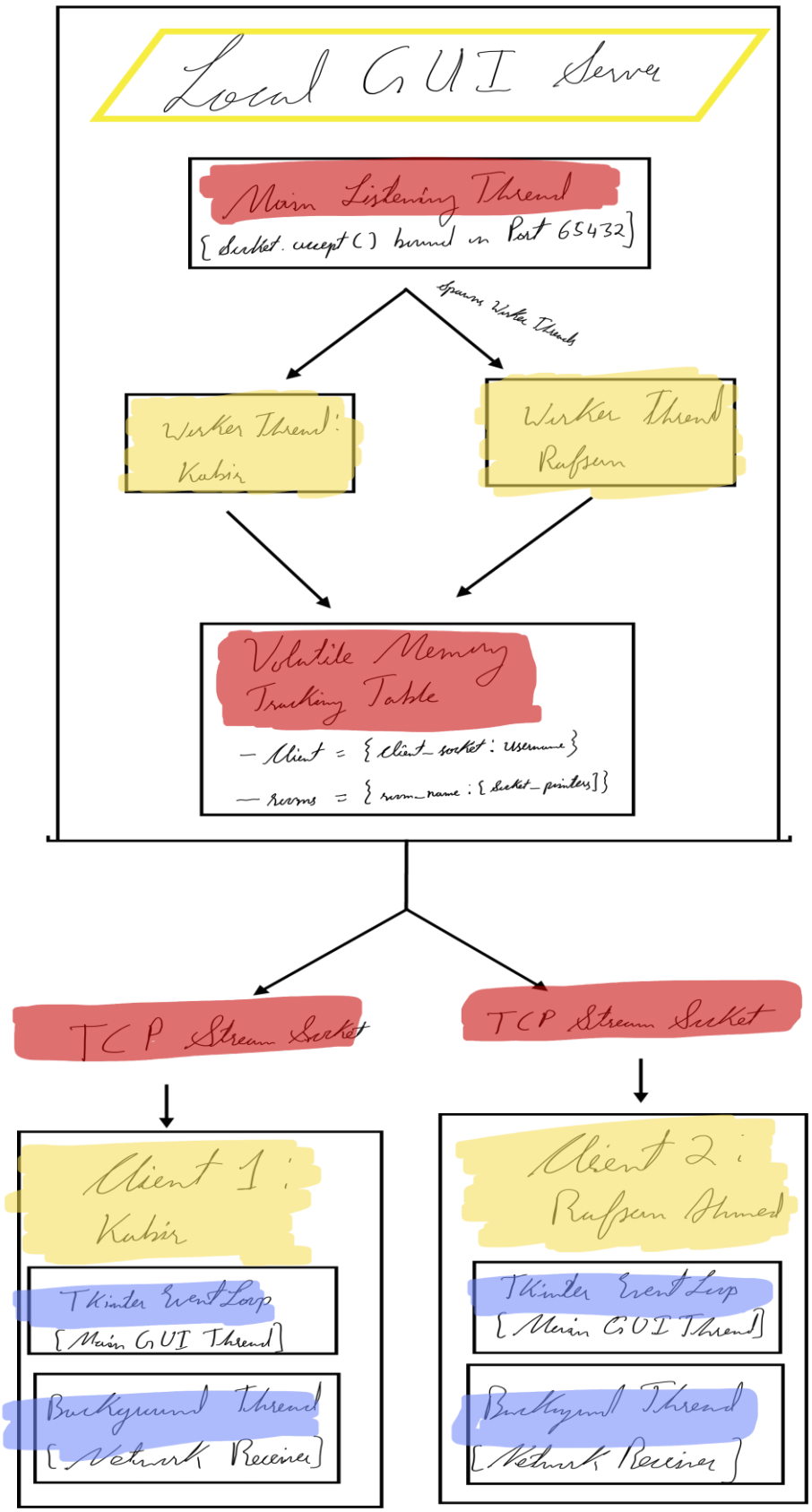
- Client (client.py): The production application endpoints for exclusive use. It uses an unsynchronized, event-driven graphics pipeline that operates over the native Tkinter layout framework. The background network data receiver thread (receive_messages) is kept uninitialized until the user completes the identity registration phase to ensure thread safety and avoid locking the visual interface during network I/O blockages.

1.2 Graphics user interface component specifications:

The GUI controls are designed to coordinate with low-level network states without causing graphical rendering blockages:

Structure parts:	Graphical interface description for client.py
User interface layer	Event driven, unsynchronized windows using native Tkinter frame pipeline.
Username input mode	Context of a modal pop-up overlay (tk.Toplevel) that asks for verification before starting chat operations.
Output presentation	Thread-safe stream ingestion appending data to a read-only, scrollable text widget area.
Transmission trigger	Clicks on event-bound button layouts or immediate hooks to the keyboard <Return> key.
Thread design	The root.mainloop() takes care of the main execution thread, while incoming data is processed in a dedicated background thread.

1.3 Structural Architecture Diagram:



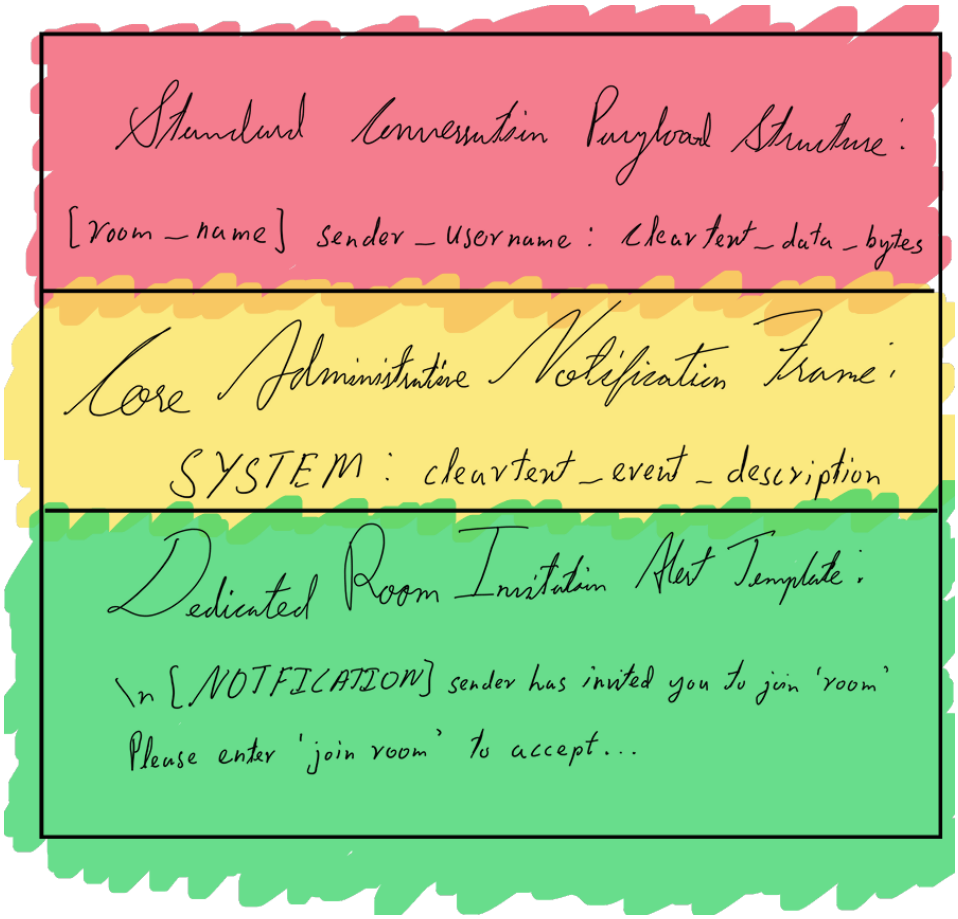
2.Protocol description:

The system uses a tailored Application-Layer Control Protocol, which is clearly layered over a reliable transport layer TCP stream. In frame processing loops, custom validation headers are checked one after the other to differentiate application-state routing events from regular visual chat text strings.

2.1 Application-Layer command dictionary:

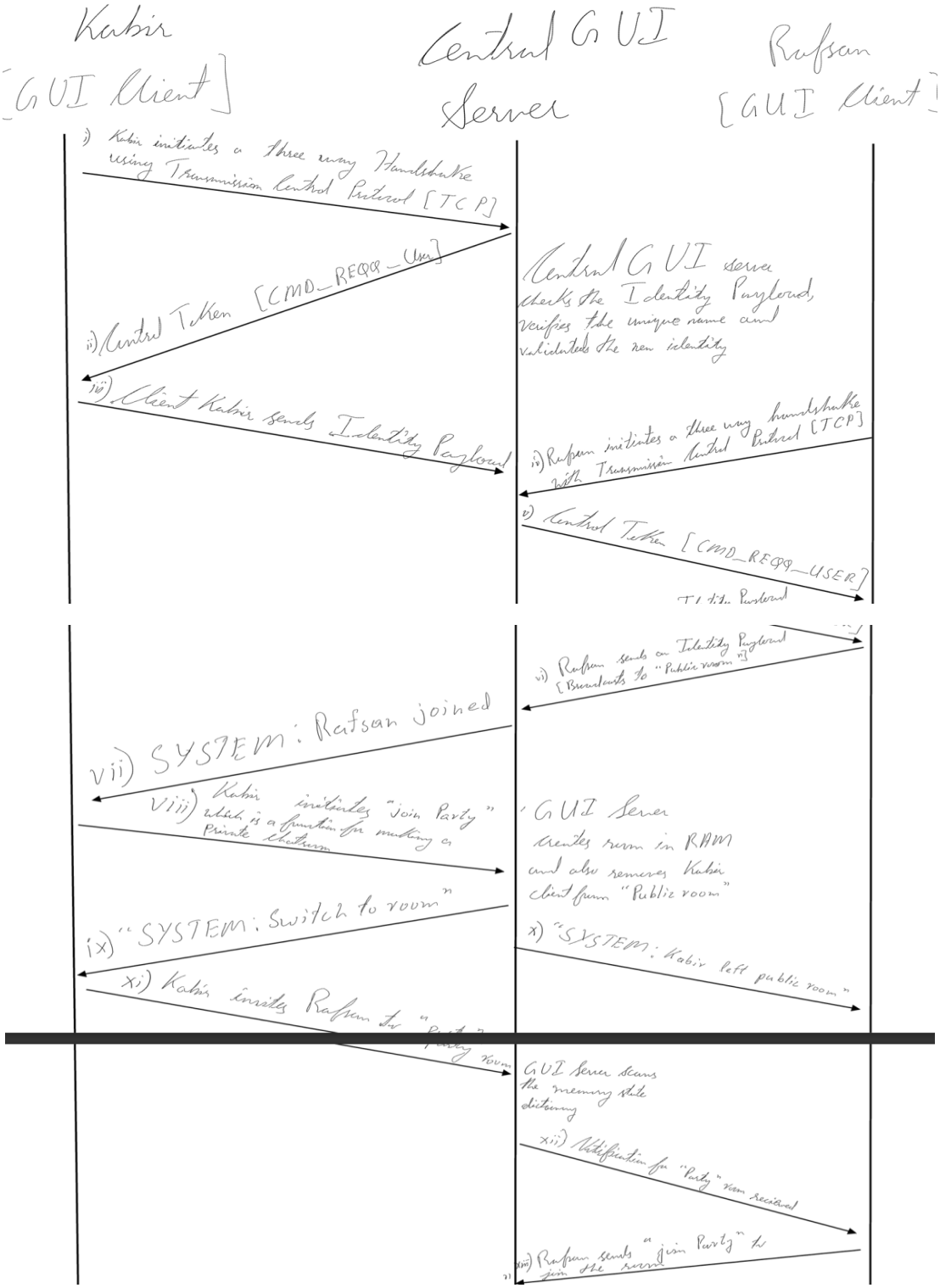
- CMD_REQQ_USER: Transmitted by the server straight to a newly bound socket endpoint to halt execution until identity parameters are supplied.
- ERR_USER_TAKEN: If the validation handler detects a duplicate username, broadcast this back to the client to trigger a visual error box and a clean socket teardown.
- join <room>: Issued from the GUI input box to modify channel tables, update server memory arrays, and automatically create a new room array if the destination is nonexistent.
- invite <username>: Prompts the server execution engine to search active memory tables for a specific user and forward an explicit join invitation alert to them.
- leave: remove a client socket entry from the current custom room tracking array, and puts them back into the public room tracking pools.
- quit: Issued using button clicks or exit strings to alert the remote server node of a pending disconnect to achieve a clean memory release.

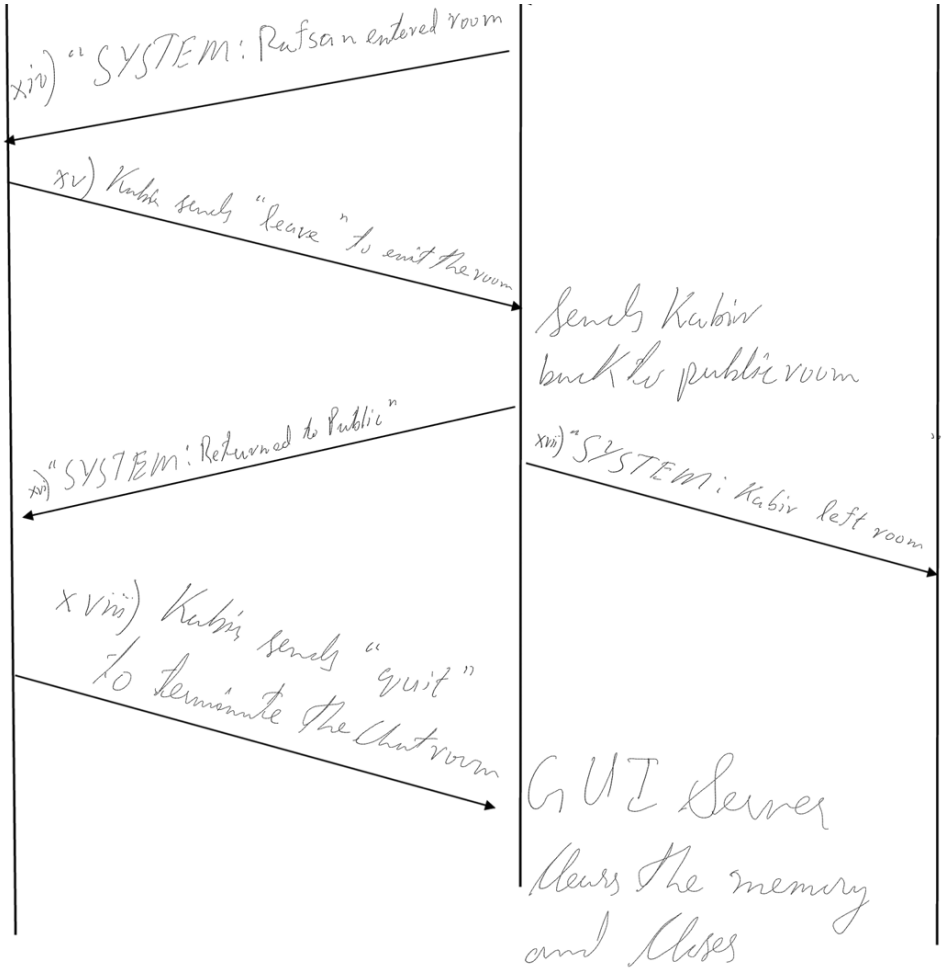
2.2 Wire frame encapsulation map:



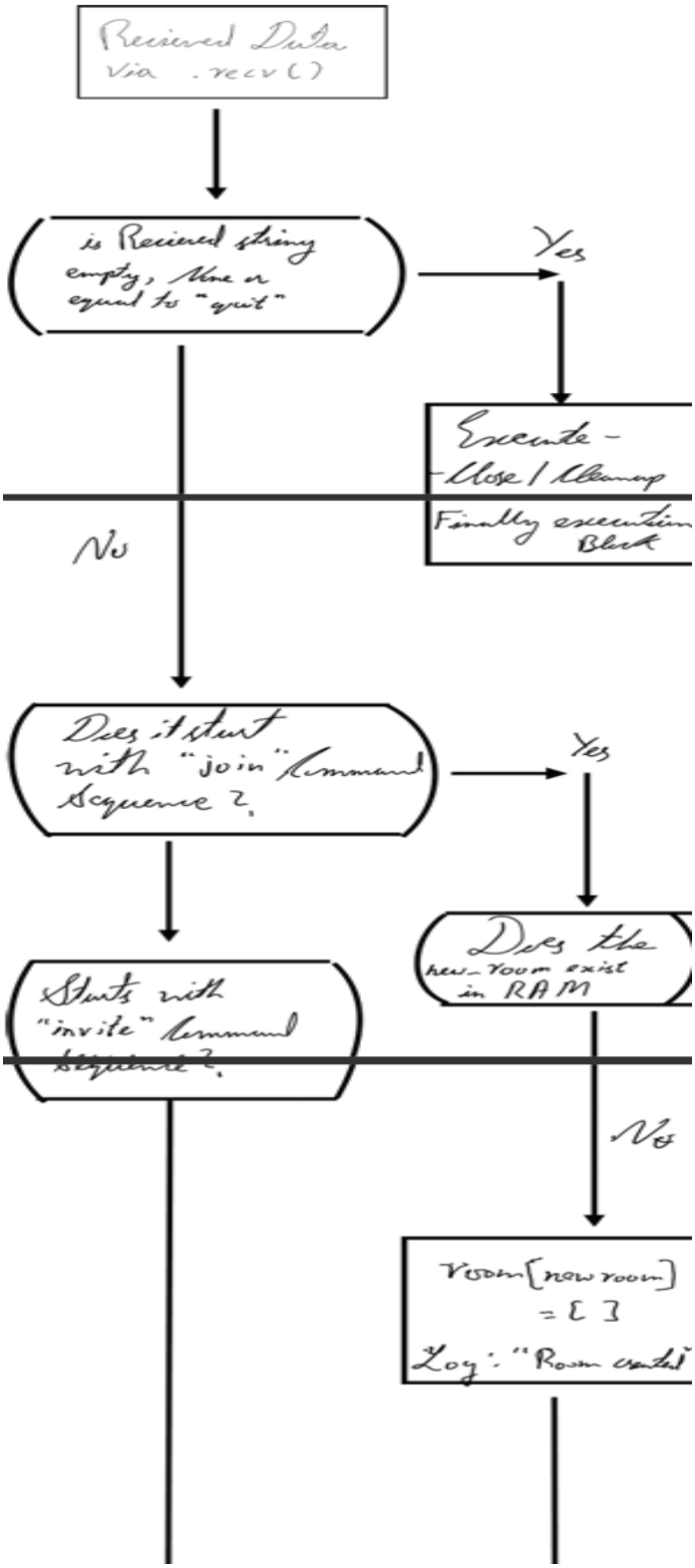
3.Network communication flow chart and finite state machine:

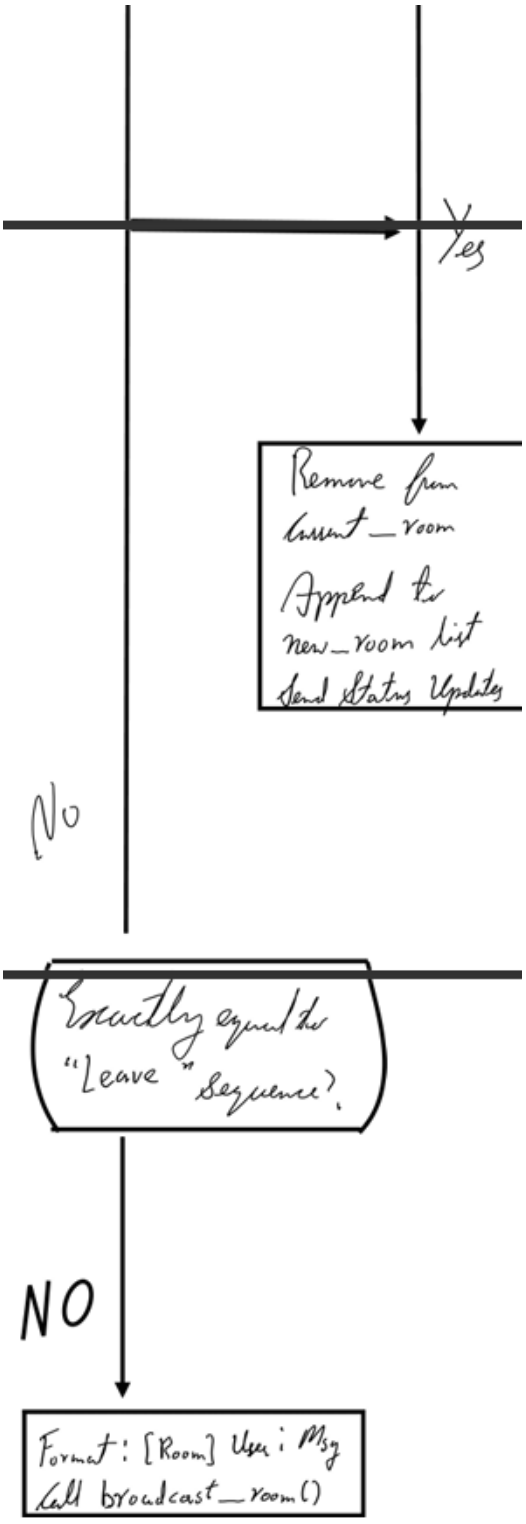
3.1 End-to-End message sequence chart:



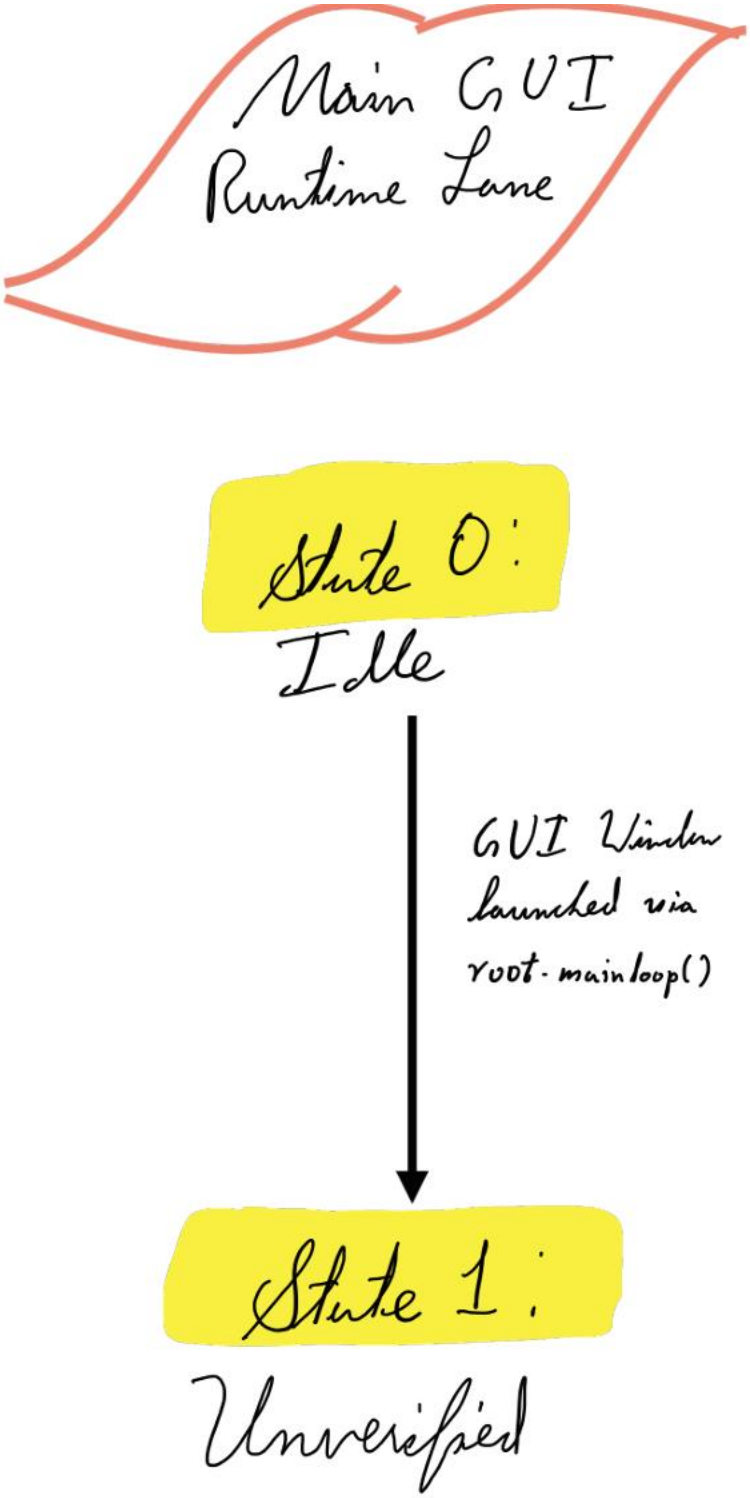

xiv) "SYSTEM: Rafsan entered room
xv) Kabir sends "leave" to exit the room
Sends Kabir back to public room
xvi) "SYSTEM: Returned to Public"
xvii) "SYSTEM: Kabir left room
xviii) Kabir sends "quit" to terminate the chatroom
GUI Server clears the memory and closes

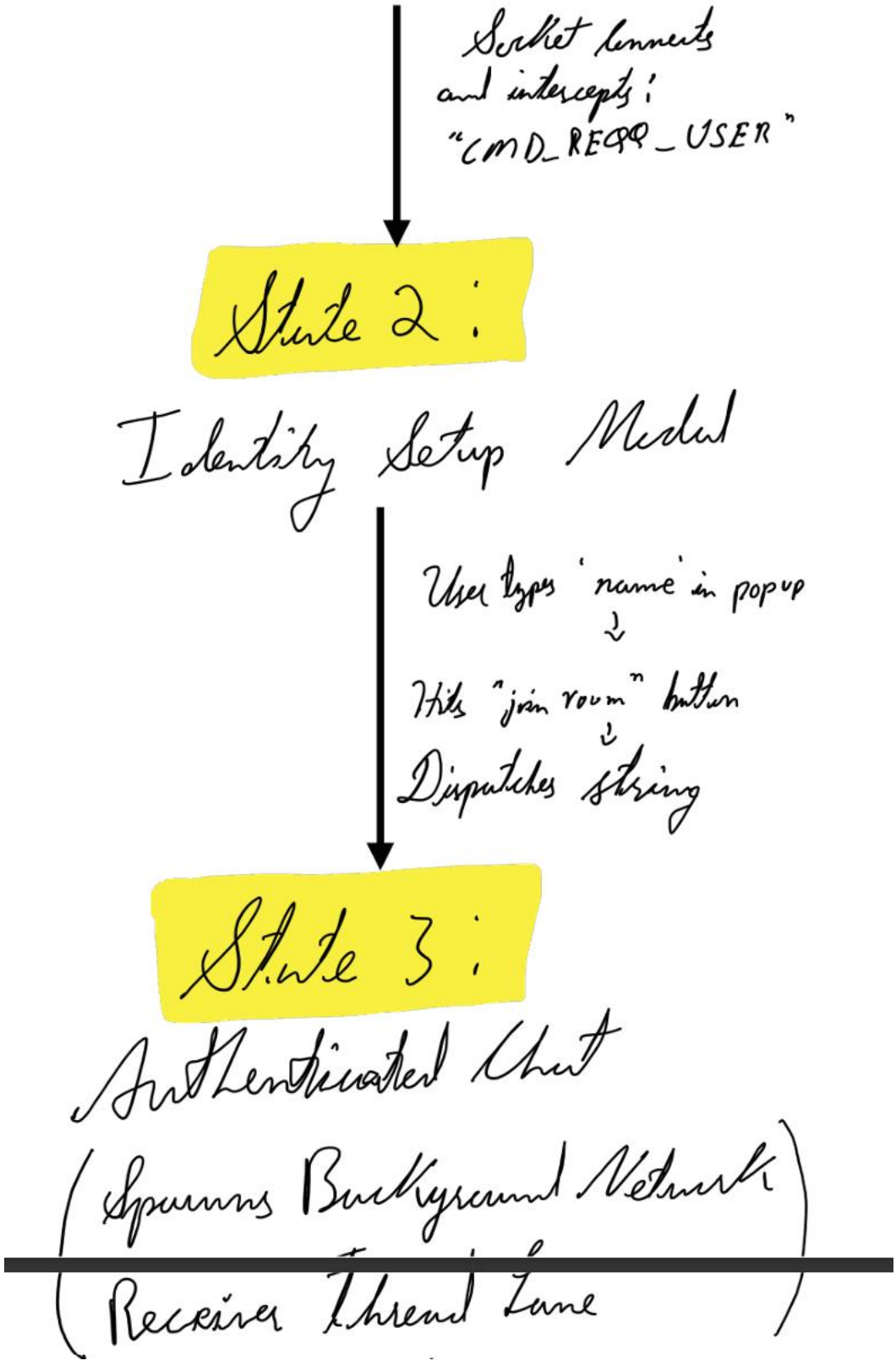
3.2 Server flow chart:

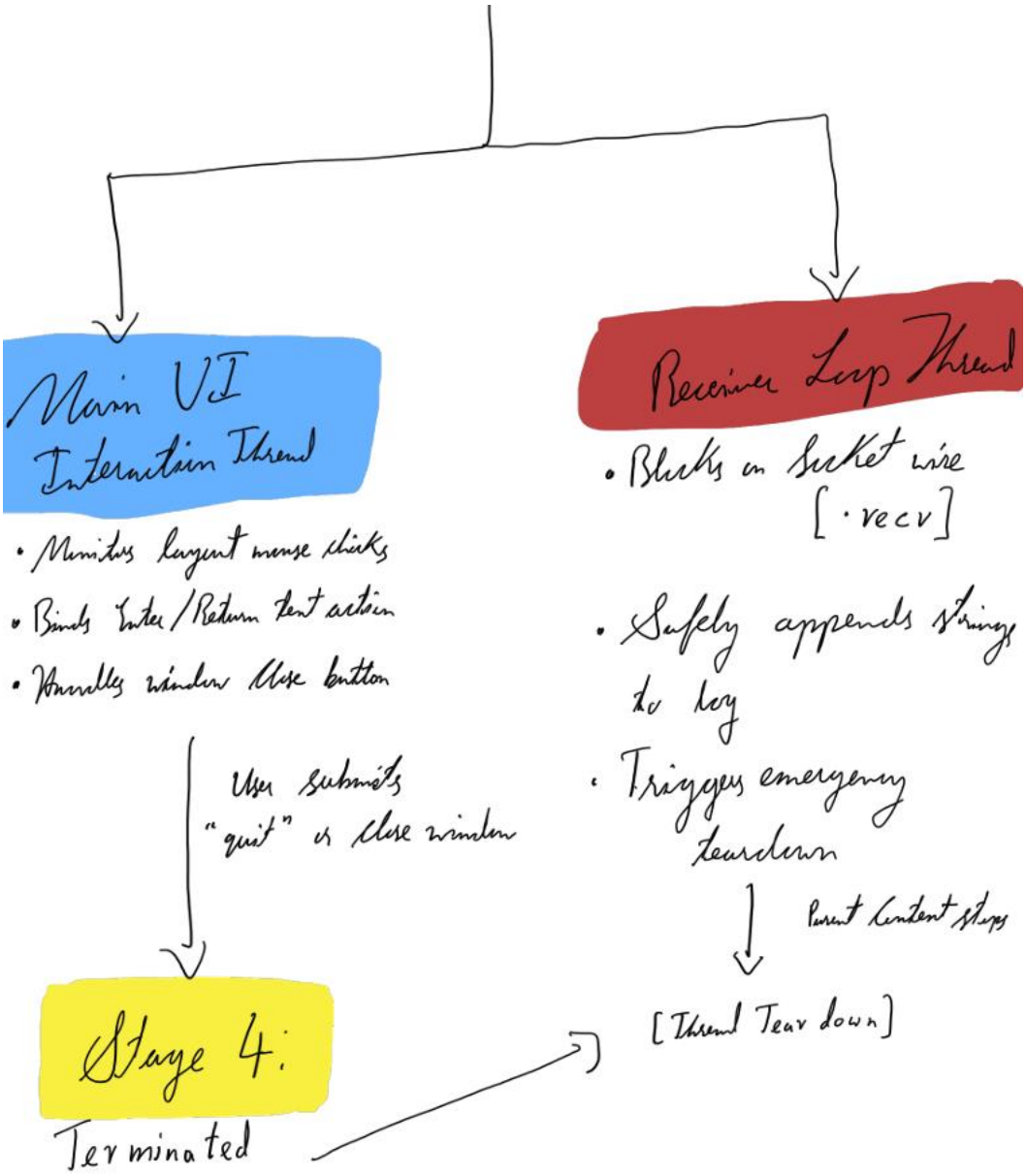



Yes
Remove from
current_room
Append to
new_room list
Send Status Updates
No
Exactly equal to
"Leave" sequence?
NO
Format: [Room] User: Msg
Call broadcast_room()

3.3 Client finite state machine:




Socket connects and intercepts: "CMD_REQ_USER"
State 2:
Identity Setup Modal
User types 'name' in popup
Hits "join room" button
Dispatches string
State 3:
Authenticated Chat
(Spawns Background Network
Receiver Thread Lane


Main UI Interaction Thread
• Monitors keyboard mouse clicks
• Binds Enter/Return key action
• Handles window close button
User submits "quit" or close window
Receiver Loop Thread
• Blocks on socket wire [.recv]
• Safely appends strings to log
• Triggers emergency teardown
Parent Content stops
[Thread Tear down]
Stage 4:
Terminated

4.Transport layer review and performance metrics:

4.1 Selecting Transport Layer Protocol: TCP or UDP

Real-time messaging applications requires session management to be structured, making TCP (socket. SOCK_STREAM) the only viable approach over UDP (socket. SOCK_DGRAM) for real-time messaging applications:

- Application Payload Typology: Text messaging applications demand a high data accuracy. Lose one TCP segment and an entire block of text is corrupted. TCP has packet retransmission built in so it does not lose your text.
- Deterministic Serialization Verification: UDP sends out data frames independently, which can cause out-of-order delivery. Before delivering to the application layer, TCP arranges streams of data in chronological order with sequence validation arrays.
- Volatile Resource Lifecycle Tracking: TCP has persistent handshake sessions, so can detect unexpected client drops by responding with empty-bytes (if not data: break), so server can clean up resources immediately.

4.2 Network Deployment Matrix:

All connection interfaces are located directly over local area adapters giving clear diagnostic visibility of this infrastructure.

Structural performance section:	Selected path: by local host TCP protocol
Data integrity verification	Absolutely. Automated retransmission parameters ensure that no frames are dropped, regardless of text logs.
Stream Ordering strategy	Sequential assembly ensures that messages are presented in the same order in the GUI log box.
State table maintenance	Connection-oriented loops don't expect immediate server dictionary cleanup at app exits.
Data encryption status	The plaintext design enables direct, step-by-step inspection of frames using network analysis tools such as Wireshark.

5.System implementation and environment setup:

5.1 Multi-computer LAN setup:

We use this to execute the application across different physical hosts devices connected to the same local area network:

1. Find the server IP Address: Run ifconfig if using Linux / macOS or ipconfig for windows on the machine that uses that will run the server file.
2. Update client.py IP connection with the private network Ip address of the device who is running the server.py. Change the ser_Ip = to the IP of the device server is on and ensure that the port stay 65432 for both.
3. Launch the server client file then make the client file run on the multiple terminals so you can have people to chat with.

GitHub link with server and client file:
https://github.com/25raf21/Comp_Network_Project_2026